

TUGAS ALGORITMA & STRUKTUR DATA

Singly Linked List (SLL)

Eksplorasi konsep, struktur node, dan implementasi senarai berkait tunggal dalam manajemen memori dinamis.

Nama Kelompok:

Amanda Ramadhani Putri Pangesti (3125500038)

Ahmad Tsani Misbahus Syafi' (3125500042)

Filo Mahabah Dienulloh (3125500058)

Konsep Dasar & Struktur

Apa itu Singly Linked List?

Struktur Dinamis

Koleksi linear dari elemen data yang disebut node. Berbeda dengan array, alokasi memori SLL bersifat dinamis—node dapat tersebar di lokasi memori yang tidak berdekatan.

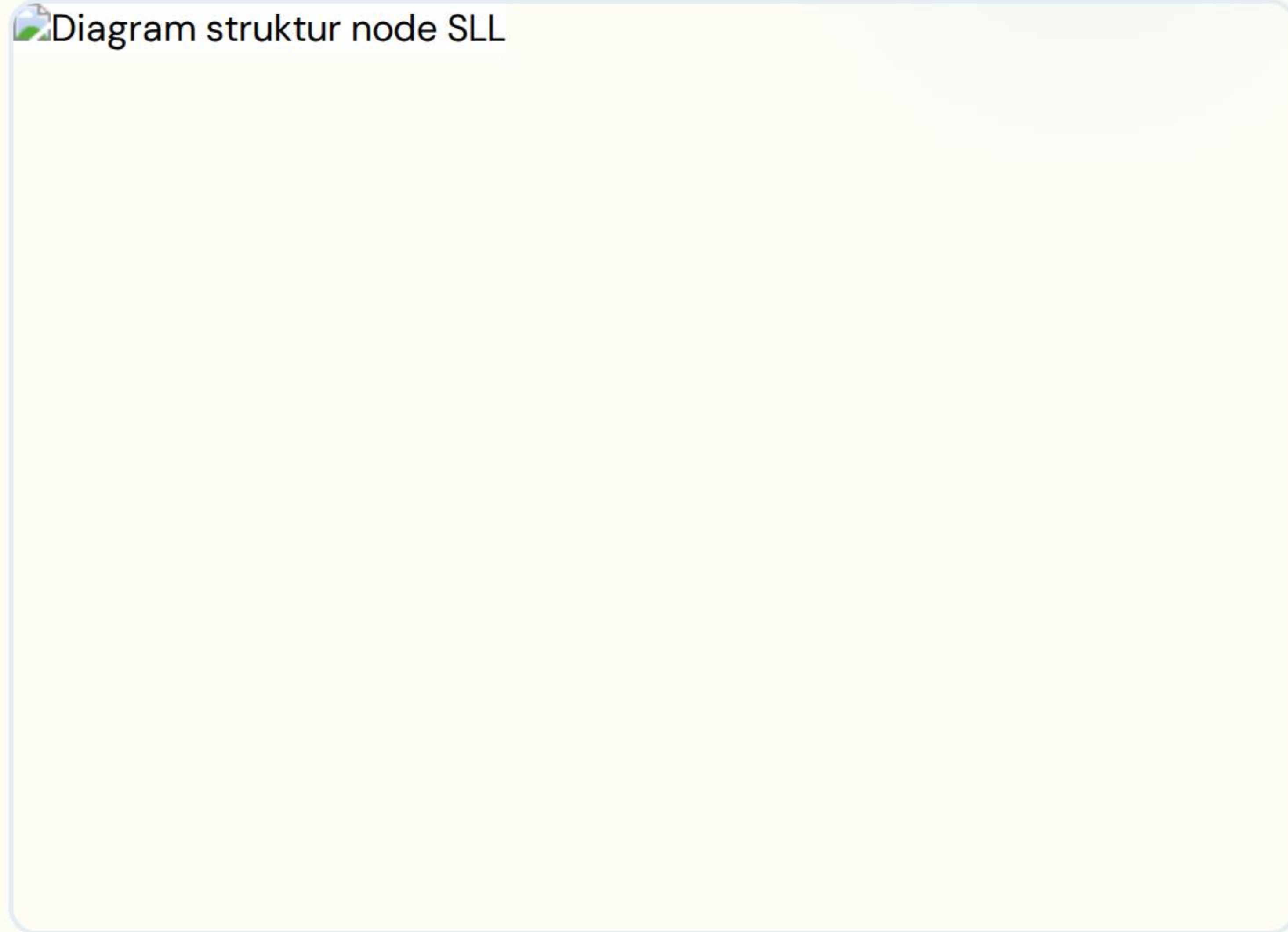
Koneksi Satu Arah

Setiap elemen hanya memiliki satu referensi (pointer) yang menunjuk ke elemen berikutnya. Jalur data bersifat sekuensial, dimulai dari Head hingga berakhir di Null.

Anatomi Sebuah Node

Node adalah unit terkecil dalam Linked List yang menyimpan informasi penting:

- **Data Field:** Menyimpan nilai atau informasi aktual (Integer, String, dll).
- **Next Pointer:** Menyimpan alamat memori dari node berikutnya dalam urutan.
- **Tail Concept:** Node terakhir selalu menunjuk ke NULL, menandakan akhir dari list.

Diagram struktur node SLL

Operasi Utama: Penambahan



Insert Front

Menambahkan node di awal list. Operasi ini sangat efisien dengan kompleksitas **$O(1)$** .



Insert Middle

Menyisipkan data setelah node tertentu. Memerlukan traversal dengan kompleksitas **$O(n)$** .



Insert End

Menambahkan node di posisi paling akhir. Memerlukan penelusuran hingga mencapai node Tail.

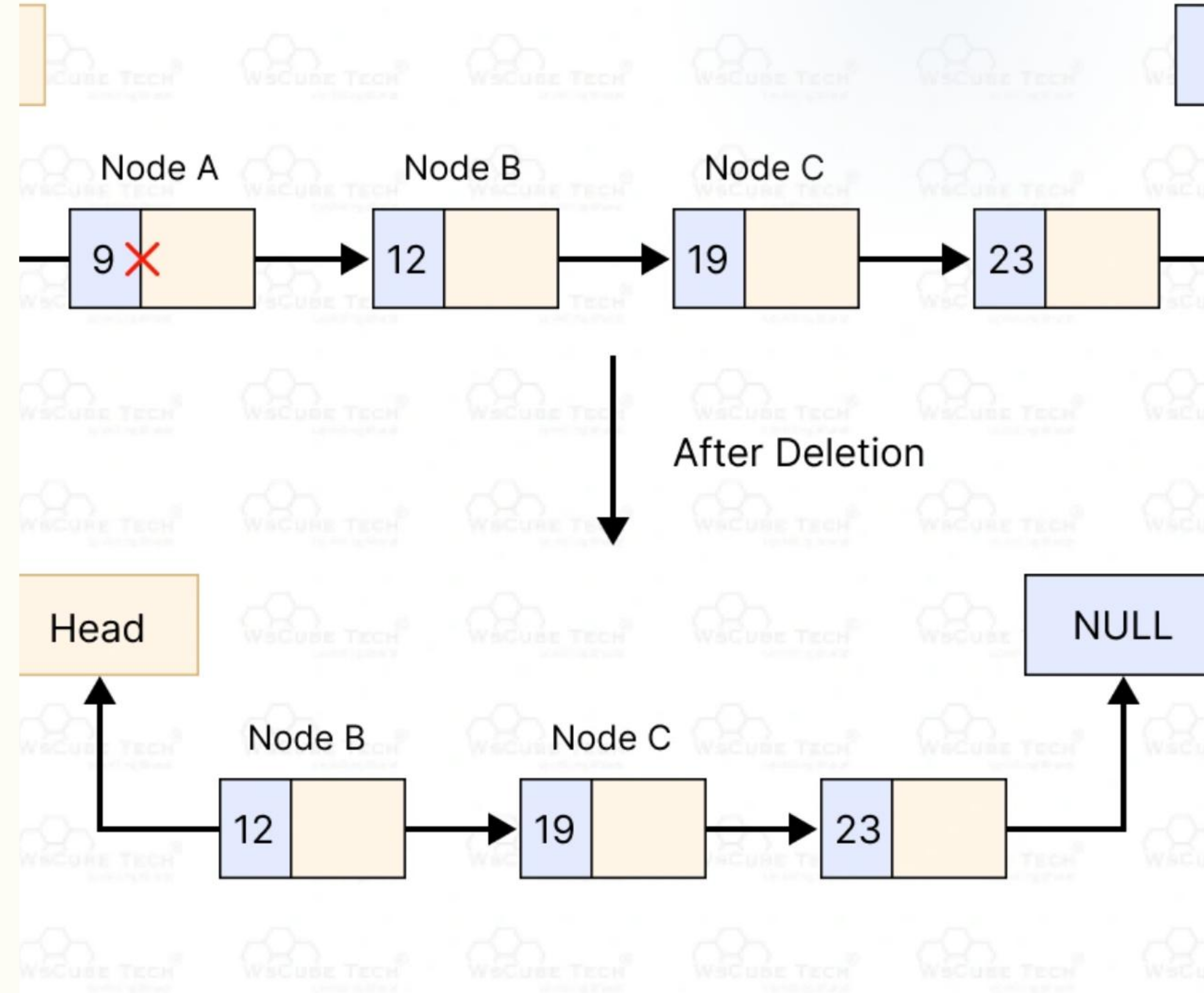
Operasi Penghapusan

Proses penghapusan elemen dilakukan dengan mengubah alur pointer agar node yang dihapus tidak lagi terhubung.

Langkah utama meliputi identifikasi node target, menghubungkan node sebelumnya ke node sesudahnya, kemudian membersihkan memori (free/delete).

"Penting untuk selalu memastikan Head tidak terputus saat menghapus elemen pertama."

Deletion At Beginning



SLL vs Array: Perbandingan

Fitur	Array	Singly Linked List
Ukuran	Statis (Tetap)	Dinamis (Fleksibel)
Akses Elemen	Cepat ($O(1)$) via Indeks	Lambat ($O(n)$) via Traversal
Insert / Delete	Lambat (Perlu Geser Data)	Cepat (Hanya Ubah Pointer)
Penggunaan Memori	Hemat (Hanya Data)	Lebih (Data + Pointer)

| Keunggulan Utama SLL

- ✓ **Efisiensi Memori:** Hanya menggunakan memori yang diperlukan saat runtime, menghindari pemborosan ruang.
 - ⚡ **Manipulasi Data Cepat:** Penambahan dan penghapusan di awal list tidak memerlukan pergeseran elemen lain.
 - 📏 **Skalabilitas:** List dapat tumbuh atau menyusut tanpa perlu melakukan alokasi ulang seluruh blok memori (reallocation).
-

Keterbatasan Struktur

Navigasi Satu Arah

Hanya bisa bergerak maju (Next). Untuk mengakses node sebelumnya, traversal harus dimulai kembali dari Head.

No Random Access

Tidak mendukung pengambilan data langsung berdasarkan indeks. Harus menelusuri satu per satu dari elemen pertama.

| Analisis Kompleksitas Waktu

$O(n)$

Searching & Access

Kompleksitas $O(n)$ muncul karena SLL harus melewati setiap node secara berurutan untuk menemukan target.

Berbeda dengan Array yang mendukung akses langsung $O(1)$, efektivitas SLL sangat bergantung pada posisi elemen yang dituju.

| Kesimpulan

// *Singly Linked List adalah pondasi struktur data dinamis yang esensial, menawarkan fleksibilitas luar biasa dalam manajemen memori dan kemudahan manipulasi data, meski harus berkompromi pada kecepatan akses acak.*

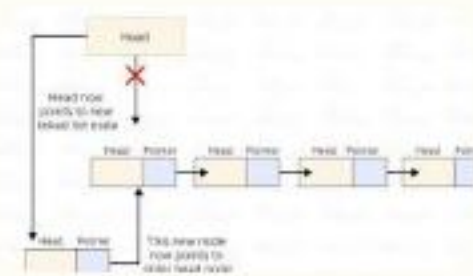
Sesi Tanya Jawab

Terima kasih atas perhatian Anda dalam presentasi Singly Linked List ini.

Dosen Pengampu: Algoritma & Struktur Data

PENS Surabaya – 2024

Image Sources



<https://deen3evddmddt.cloudfront.net/uploads/content-images/singly-linked-list-operations-in-data-structure.webp>

Source: www.wscubetech.com



Thumbnail for <https://deen3evddmddt.cloudfront.net/uploads/content-images/deletion-at-the-beginning.webp>

Source: www.wscubetech.com